Title: Event Booking Platform

Pitch: A full-stack app that centralizes event discovery and ticket tracking, letting users browse events by date and view ticket availability while admins manage listings through a REST API.

- This is my capstone for UCI 2123.
- It’s a full-stack build: React frontend, Spring Boot backend, MySQL database.

Problem & Solution (1 minute) (todo: remove timings)

Problem (before):

- Event information is often fragmented (spreadsheets, posts, multiple tools).
- Users struggle to find events by date/status and understand availability.

Solution (after):

- Centralized event catalog with calendar discovery.
- REST API supports filtering (date range, status), pagination, and consistent error handling.
- Ticketing represented as event inventory and completed sales records for reporting/auditing.

- The core user experience is: discover events quickly, view details, and proceed toward booking.
- The core admin experience is: manage event lifecycle and data consistency through rules (immutability where appropriate).

Architecture Overview (2 minutes) TO DO

Architecture (high level):

- **React** frontend (component-based UI, routing, API integration)
- **Spring Boot** REST API (controller/service/repository layers)
- **MySQL** relational database (tables + constraints)

```
[ React Frontend ]
      |
   HTTP / REST
      |
[ Spring Boot API ]
      |
   JPA / Hibernate
      |
[ MySQL Database ]
```

- Show ERD for MySQL database – Explain the relationships between the tables
- Frontend calls the API for event data and renders a calendar/list experience.
    - E.g. http://localhost:8080/api/events
- Backend enforces business rules, validation, pagination, and consistent REST patterns.
    - Fill out data here
- Database persists events, venues, users, tickets, and ticket sales for reporting.
    - Fill out data here

Slide 4 — Live Demo Preparation (transition)

- App is running locally.
- Demo will show core user flows and the API-backed UI.
- Setup steps are documented in README (backend + frontend).

- I'll walk through the main routes and show how UI actions map to API responses.

Slide 5 — Live Demo (3 minutes)

- **Demo flow (recommended order):**
- **Home / Calendar route**: show month navigation and event discovery.
- **Event detail view** (modal/dialog): show full event name/date/status and the "Buy Ticket" button (even if not wired yet).
- **Events list route**: show pagination/sorting/filtering working from the API.
- **My Tickets route**: show user-facing ticket history/ownership concept.
- **Admin Manager** Show admin management and displays events with sold out tickets.

- [Put recorded video here showing off the Event Booking Application, once stable, in case it breaks on presentation day]

SLIDE 6 – TO DO - Slide 6 — Technical Deep Dive (2 minutes

Slide 7 — Security & Local Setup (1 minute) - TO DO

Slide 8 — Lessons Learned (1 minute)

- **Lessons learned:**
  - Shipping iteratively beats over-polishing one screen.
  - Clear scope boundaries prevent regressions and keep progress measurable.
  - Accessibility and responsiveness require testing across constraints, not assumptions.
  - Separating concerns (UI components vs API services vs DB rules) makes debugging and enhancement safer.
  - AI really does 10x performance, but the generated code must be checked and made sure it works, as there will be errors at times.

AI tools were used responsibly to improve productivity; I reviewed all generated code and maintained ownership of design decisions.

Slide 9 — Q&A

- Questions?